

Credence-Core: An Institutional Digital Banking and Wealth Management

Comprehensive Technical Project Report

Nigam Mehta

mehtanigam3024@gmail.com

Abstract—Credence-Core is a full-stack digital banking and wealth-management simulation developed as a college engineering project. The application combines a C++ core ledger engine, a Python Flask middleware layer, and a browser-based interface implemented with HTML, modern JavaScript, and Tailwind CSS. The system models account creation, customer identification, deposits and withdrawals, transfers, loans, fixed deposits, UPI registration, debit-card services, cheque-book dispatches, transaction history, analytics, investment portfolios, systematic investment plans, digital gold, and certified PDF statements. The repository also contains Render, Docker, Vercel, and CMake configuration intended to support multiple deployment modes. This report analyzes the repository structure, runtime architecture, data model, API layer, security mechanisms, feature workflows, deployment configuration, installation procedure, current limitations, and future engineering scope. The analysis is grounded in the project repository and the supplied application screenshots. At the time of preparation, the public Render URL returned a service-unavailable response, so live-runtime claims are limited to repository evidence and the supplied screenshots.

Index Terms—Digital banking, fintech simulation, C++, Python Flask, REST API, Tailwind CSS, JSON persistence, wealth management, SIP, digital gold, PDF reporting, Render, Docker.

I. PROJECT OVERVIEW

A. Purpose and Motivation

Credence-Core is designed as an institutional-style digital banking terminal rather than a single-purpose transaction program. The repository describes the application as a full-stack digital banking, wealth-management, and core-ledger platform, combining a high-performance C++ ledger backend with Python Flask middleware and a modern browser interface. The design deliberately separates financial operations from presentation logic so that the same core operations can be exposed through a local executable and a web application. [1]

The project addresses an educational engineering objective: demonstrate how a traditional transaction-processing core can be wrapped by a web API and presented through a modern financial dashboard. It therefore combines concepts from C/C++ programming, data structures, file persistence, web development, REST-style communication, financial calculations, UI engineering, and deployment.

B. Core Capabilities

The implemented feature set can be grouped into five functional domains:

TABLE I: Credence-Core Technology Stack

Layer	Technologies / Role
Core ledger	C++20/C++ source modules; account, transfer, loan, FD, UPI, cheque and reporting logic
Middleware	Python 3.11, Flask; HTTP routes, JSON serialization, C++-style backend abstraction and application orchestration
Frontend	HTML, modern JavaScript, Tailwind CSS; single unified banking command deck
Persistence	JSON files for profiles, investments and transaction-related state; in-memory C++ structures during execution
PDF reporting	ReportLab
HTML processing	BeautifulSoup4
Production server	Gunicorn
Build	CMake and direct g++ compilation
Deployment	Render web service, Docker; repository also contains Vercel configuration

- 1) **Core banking:** account creation, account classification, customer ID generation, balance management, deposits, withdrawals, transfers, loan processing, fixed deposits, and account reporting.
- 2) **Payments and services:** UPI registration, inter/intra-account transfers, utility-payment UI, cheque-book ordering, physical-card ordering, and dispatch tracking.
- 3) **Cards and security:** debit-card visualization, CVV/expiry display, card blocking and PIN-oriented interactions, with balance masking in the UI.
- 4) **Wealth management:** fixed deposits, mutual-fund/SIP workflows, IPO/RD portfolio structures, and digital-gold tracking.
- 5) **Reporting and analytics:** transaction ledger, cash-flow analysis, portfolio summaries, and PDF statement generation through ReportLab.

C. Technology Stack

The repository's `requirements.txt` contains Flask, BeautifulSoup4, ReportLab, and Gunicorn. The CMake configuration targets C++20 and produces the `bank_system` executable. [4], [5]

D. Development-Project Scope

The repository explicitly identifies Credence-Core as a college development project intended for educational and demonstration purposes. It instructs users not to enter real banking

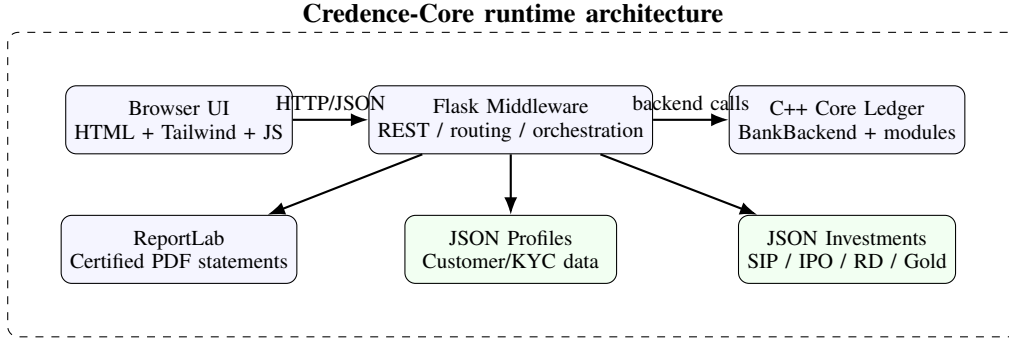


Fig. 1: Logical architecture derived from the repository: browser presentation, Flask middleware, C++ banking core, JSON persistence, and PDF reporting.

credentials, passwords, PINs, card details, or other sensitive information and states that it is not connected to a real banking institution. This boundary is important because several screens intentionally resemble production banking interfaces while the underlying persistence and authentication mechanisms remain simulation-oriented. [1]

II. TECHNICAL DETAILS

A. Backend Setup and Dependencies

The Python application is centered on `flask_app.py`. It imports Flask request/response utilities, BeautifulSoup4 for HTML post-processing, ReportLab for PDF generation, and standard Python modules such as `json`, `hashlib`, `random`, `re`, `subprocess`, and `threading`. [2]

The backend also maintains a `BankBackend` abstraction. Its execution layer models operations such as account creation, balance checking, transfers, loans, FD creation/withdrawal, reporting, UPI registration, and cheque operations. The repository uses a C++ source set for the native core and a Python-side abstraction for web execution.

B. Frontend Structure

The web UI is divided conceptually into an authentication/landing page and a unified banking command deck. The repository maps routes such as `/dashboard`, `/account`, `/upi`, `/loans`, `/transactions`, `/analytics`, `/reports`, `/settings`, `/investments`, `/transfer`, and `/cards` to the unified HTML command deck. The Flask layer also dynamically ensures that `/app.js` is attached to served HTML when required. [11]

The frontend controller in `app.js` synchronizes account data, analytics, loans, investments, dispatch information, and transaction history. For example, account synchronization requests `/api/account/<accNo>`, while analytics and loan views use `/api/analytics/<accNo>` and `/api/loans/<accNo>`. The SIP workflow posts to `/api/invest/sip`. [3]

C. Persistence Model

Credence-Core does not use a relational SQL schema in the visible repository. Instead, it combines in-memory

C++ account structures with JSON persistence managed by the Flask layer. The README identifies local transaction/profile/investment stores, and the current source initializes investment records using a JSON file. This approach reduces setup complexity and is appropriate for a college-scale simulation, but it introduces important durability, concurrency, and multi-instance limitations for production use.

The main logical entities are:

- **Account:** account number, name, balance, loan amount, fixed deposit and transaction history.
- **Customer profile:** 9-digit customer ID, account type, email, date of birth, address and authentication-related fields.
- **Card:** card number, expiry, CVV, provider, PIN-related state and card-service metadata.
- **Investment portfolio:** SIPs, IPOs, recurring deposits, and gold holdings.
- **Dispatch record:** cheque-book and physical-card order/tracking information.

D. C++ Core Modules

The C++ core is intentionally modular. `account.cpp` handles account creation, account-number generation, balance checks and transfers. The account generator creates a unique 14-digit number using a random engine and checks for collisions before returning the value. [8]

`credit.cpp` and `debit.cpp` represent money-in and money-out operations; `fd.cpp` implements fixed-deposit creation, withdrawal and maturity estimation; `loan.cpp` handles loan workflows; `upi.cpp` models UPI registration and related logic; `cheque.cpp` contains cheque-book management; `report.cpp` supports account reporting; and `utils.cpp/globals.cpp` provide shared helpers and state. The executable entry point is `main.cpp`. The README presents the same module mapping at repository level. [1], [8], [9]

E. Representative C++ Account Logic

A short excerpt of the account-number generation logic illustrates the core’s design philosophy:

Listing 1: Representative account-number generation logic

```

long long generateUniqueAccountNumber() {
    static random_device rd;
    static mt19937_64 rng(rd());
    uniform_int_distribution<long long> dist(
        1000000000000LL, 9999999999999999LL);
    long long newAccNo;
    do {
        newAccNo = dist(rng);
    } while (accounts.find(newAccNo) != accounts.end()
        || newAccNo == SYSTEM_ACCOUNT_NO);
    return newAccNo;
}

```

This mechanism provides a 14-digit simulated account identifier while avoiding the reserved system account. The implementation is suitable for demonstration, but a production banking system would require a transactional database constraint rather than application-level random collision checking.

F. Flask API Layer

The Flask layer exposes both UI routes and JSON endpoints. The documented/observed endpoints include the following.

The login endpoint checks the submitted account number, loads the report, compares a stored net-banking password when one exists, and returns account number, name, balance, loan amount and fixed deposit values. Account creation additionally generates a random 9-digit customer ID and initializes a zero-investment portfolio. [10]

G. Authentication and Authorization

Authentication is currently simulation-oriented. The login request carries an account number and password to `/api/login`; the Flask layer compares the password against the stored `nb_pass` field when present. The browser stores the current account in `sessionStorage`. Logout clears the browser session and redirects to the landing page. [10], [3]

The interface also masks the account balance and provides a PIN-protected visibility toggle. However, the current JavaScript implementation performs the balance-PIN comparison in the client and includes a default 1234 fallback. This is a major limitation: it should be described as a UI privacy feature rather than a security boundary. A production design should authenticate sensitive actions on the server, never ship authoritative PIN material to the browser, and use a proper session/token mechanism with server-side authorization. [15]

H. Security and Privacy Considerations

Several useful educational controls are present: UPI PINs can be hashed with SHA-256 before storage in the profile structure, the balance is hidden by default, and the project explicitly warns against entering real credentials. [12], [15], [1]

Nevertheless, the repository also exposes security weaknesses that are expected in a demonstration system:

- 1) Net-banking passwords are stored in profile data rather than being represented by a production-grade password-hash workflow.
- 2) The client-side PIN check is not an authoritative authorization mechanism.

- 3) The Flask application enables permissive CORS headers with wildcard origin.
- 4) Card numbers, CVVs and expiry values are represented as application data for simulation.
- 5) JSON files do not provide transactional isolation, row-level locking, audit trails or strong concurrent-write guarantees.
- 6) The repository contains demo-account initialization logic, reinforcing that the application is a test environment rather than a banking service.

III. CODEBASE ORGANIZATION

A. Repository Layout

The repository is relatively flat, with the core source modules and deployment files at the project root. The principal organization is summarized in Table III.

B. Build Configuration

The CMake file declares C++20 and includes the core modules in a single executable target. [5]

Listing 2: CMake configuration pattern used by the repository

```

cmake_minimum_required(VERSION 3.20)
project(bank_system)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED True)
add_executable(bank_system
    main.cpp globals.cpp utils.cpp account.cpp
    fd.cpp upi.cpp debit.cpp loan.cpp report.cpp credit.cpp)

```

The repository also uses a direct compiler invocation in its Render configuration. This is useful because the deployment environment can build the native core without requiring a local precompiled Windows executable.

IV. FEATURES AND FUNCTIONALITY

A. Authentication and Account Creation

The first user journey begins at the institutional landing/authentication screen. The supplied screenshot shows a banking-style landing page with an account-access panel and development notice. The interface is intentionally styled as a premium banking terminal while remaining a simulation.

Account creation passes customer details to the Flask layer. The backend invokes the banking engine, extracts the generated account number, optionally registers UPI, generates a 9-digit customer ID, stores profile information, initializes investment state, and records an initial deposit. [12]

B. Dashboard and Account Overview

The dashboard consolidates the account balance, account card, cash-flow visualization, recent transactions, loan snapshot, investment information, and quick actions. The frontend loads account data first, then synchronizes analytics, loans and investments, and finally renders transaction and dispatch information. [3]

TABLE II: Representative Credence-Core HTTP API Surface

Method	Endpoint	Purpose
POST	/api/login	Validates an account number against stored net-banking password data and returns account summary information.
POST	/api/create_account	Creates an account, optionally issues a debit card, registers UPI, creates a customer ID, initializes an investment portfolio and records the initial deposit.
GET	/api/account/<accNo>	Returns account, customer-profile, card, cheque-book and physical-card information for the selected account.
GET	/api/analytics/<accNo>	Supplies inflow, outflow, balance, FD and loan analytics used by the dashboard.
GET	/api/loans/<accNo>	Supplies active/past loan information for the loans view.
GET	/api/investments/<accNo>	Supplies portfolio summary, SIP data, gold holdings and market-fund information.
POST	/api/invest/sip	Creates an SIP/investment instruction after frontend validation of the minimum amount.
GET/POST	Additional service routes	The C++ execution layer models transfers, loans, FD, UPI and cheque operations; the exact web route surface should be treated as implementation-specific and verified against the current deployment before production use.

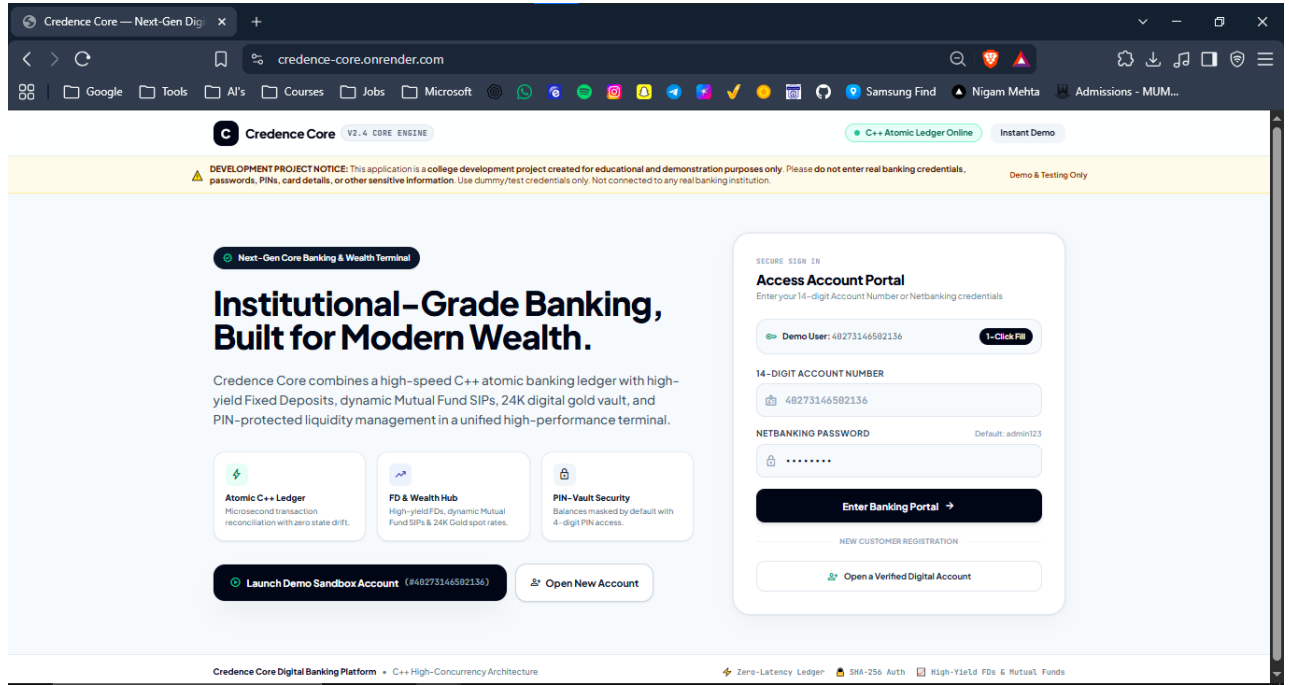


Fig. 2: Supplied Credence-Core landing and authentication screen. The interface combines institutional branding, an access panel, and a prominent development-project notice.

C. Ledger and Transaction History

The ledger view presents the user’s financial history in a tabular format. The application records events such as initial deposits and transfer activity and exposes transaction history through the account data returned to the frontend. The design is consistent with the reference reports’ emphasis on traceability: each financial operation should produce an observable record rather than changing only a balance counter.

D. Cash-Flow Analytics

The analytics layer derives total inflows, total outflows, available balance and net cash flow. The frontend computes net cash flow as total deposits minus total withdrawals and separately displays fixed-deposit and loan values. [13]

E. Fund Transfers and Payment Desk

The transfer interface models account-to-account fund movement. At the C++ core level, a transfer checks both accounts and available balance, debits the sender, credits the receiver, and records transfer-out and transfer-in events. [8]

F. Loans

Loan processing is modeled as a balance-affecting operation. The backend abstraction can approve a loan by increasing the account’s loan amount and crediting the corresponding amount to the balance; repayment reduces the outstanding loan, bounded by the current balance and outstanding principal in the demonstration model. [16]

G. Fixed Deposits

The FD module supports creation, withdrawal and maturity estimation. The C++ implementation transfers money from the

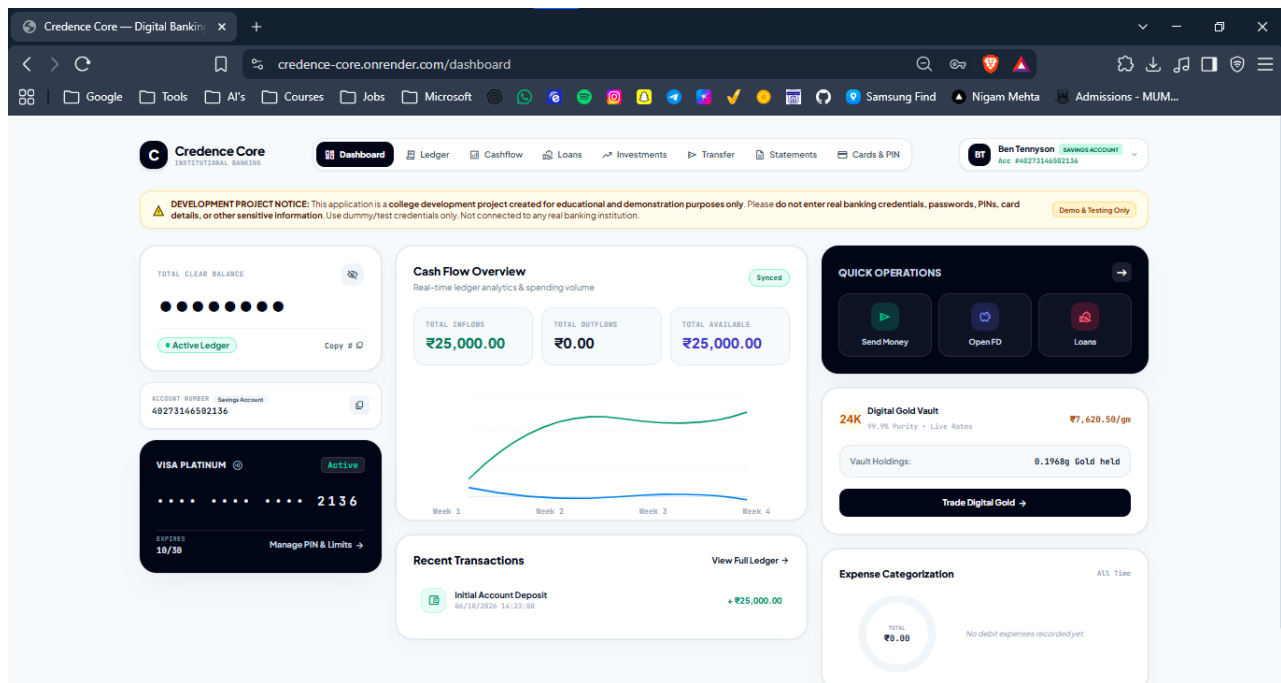


Fig. 3: Credence-Core dashboard screenshot supplied with the project. The screen acts as the main operational summary for account balance, cash flow, cards, quick operations and recent activity.

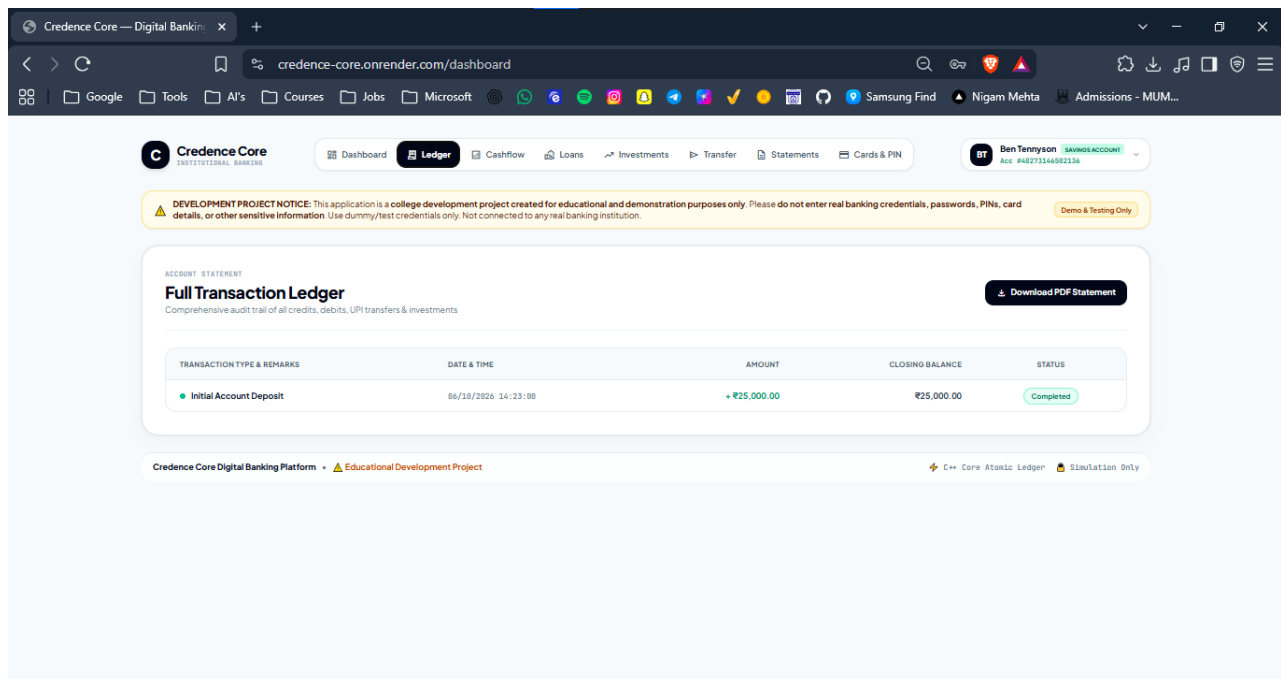


Fig. 4: Transaction ledger interface. The screenshot illustrates the dedicated history view used to inspect recorded financial activity.

account balance to the fixed-deposit field and adds a transaction entry. The Python-side execution model also supports maturity calculation using a demonstration rate schedule. [9], [16]

H. Wealth and Investments

The investment subsystem is one of the more substantial differentiators of Credence-Core. The repository initializes each new account with empty SIP, IPO and RD arrays plus a gold object containing grams, average buy price, total invested

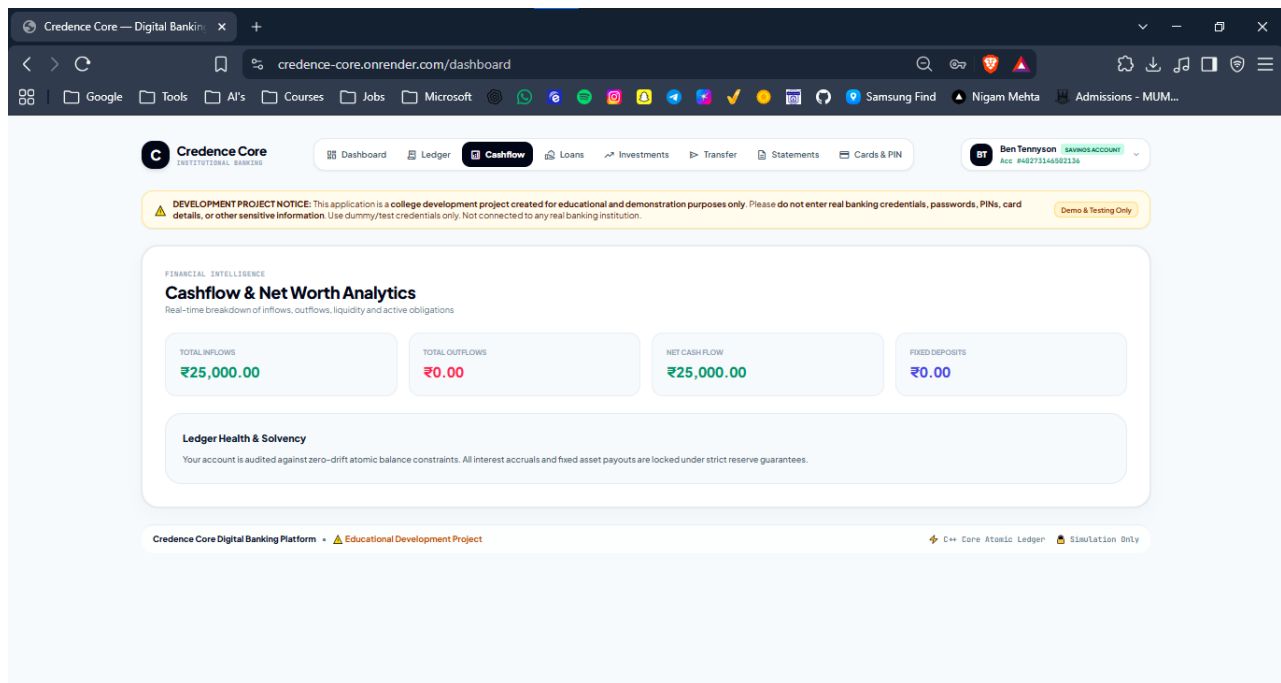


Fig. 5: Cash-flow and net-worth analytics screen supplied with the project. The view emphasizes aggregate financial indicators rather than individual transactions.

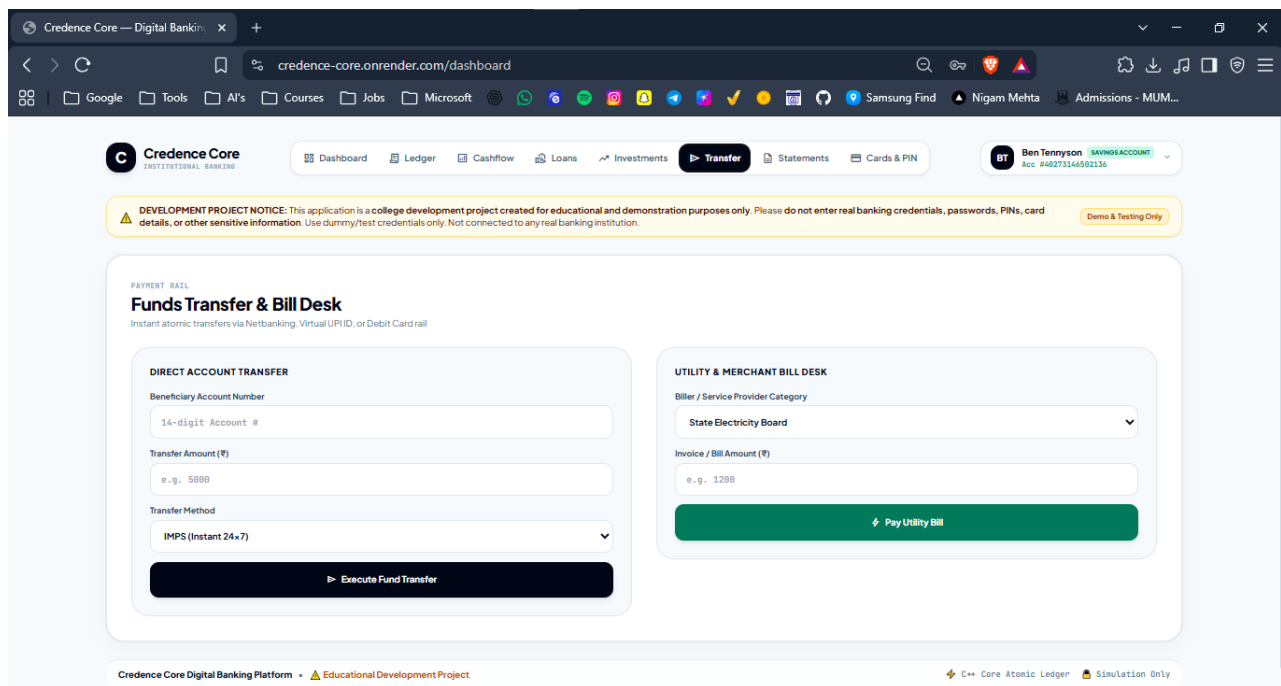


Fig. 6: Funds Transfer and Bill Desk interface. The supplied screen presents transfer inputs alongside payment/service operations.

amount, current rate, current value and return percentage. [12]

The frontend displays market funds, current NAV, category, indicative CAGR, rating and minimum SIP amount. It validates the minimum amount before sending an investment request to `/api/invest/sip`. [14]

I. Cards and Security Services

The cards screen visualizes a debit/credit-card style object and provides service controls. The backend generates simulated card metadata, while the frontend exposes a banking-style interface for card management. The repository explicitly

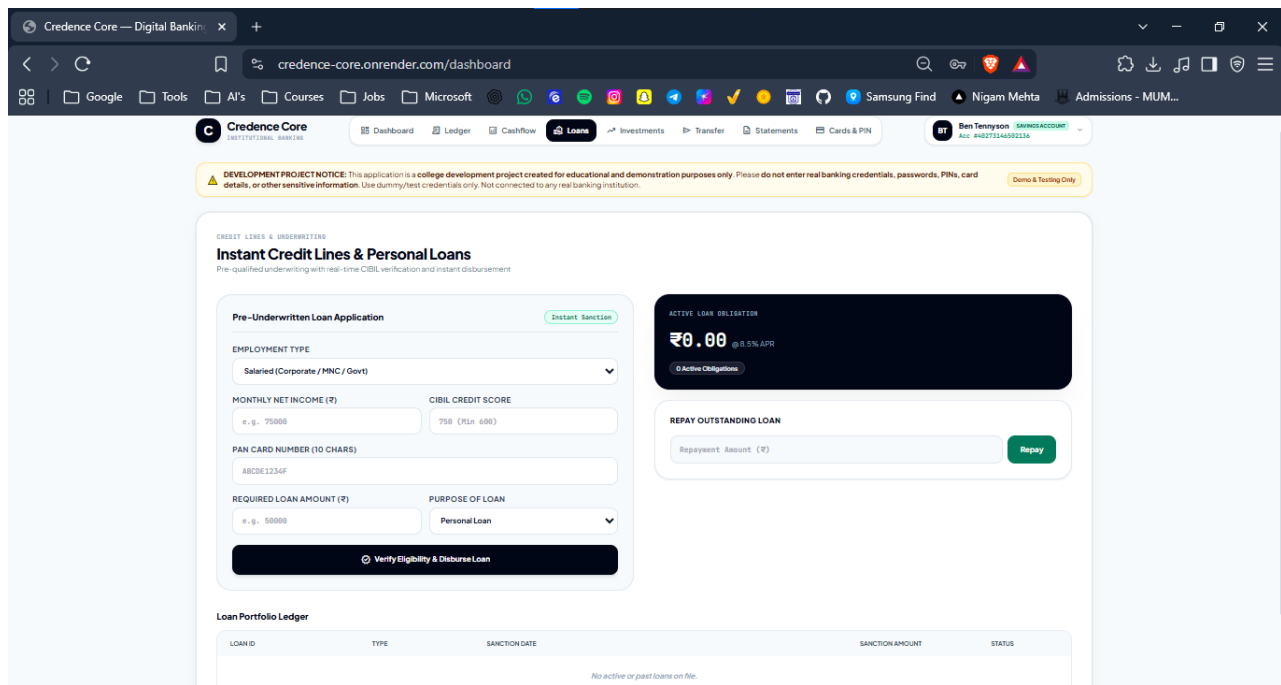


Fig. 7: Loan portfolio and personal-loan interface. The screen separates loan input/processing from the portfolio history view.

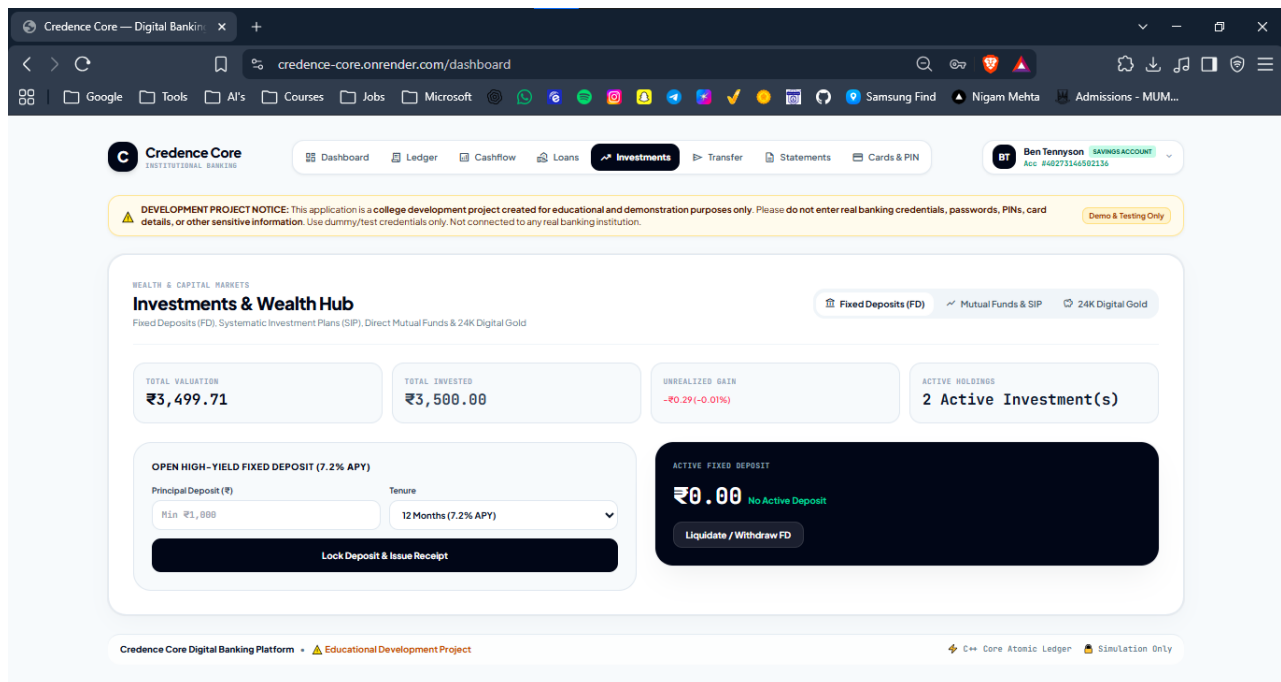


Fig. 8: Investment and Wealth Hub overview. The screenshot shows portfolio KPIs and the investment-entry area.

treats these values as demonstration data and should not be used with real card information.

J. Statements and PDF Reporting

The Flask application imports ReportLab's document, table, paragraph, style and page-size components. This supports generation of formal PDF statements rather than only HTML

tables. The repository describes the output as a certified-style bank statement containing account identity, customer information, account classification and debit/credit breakdown. [1], [2]

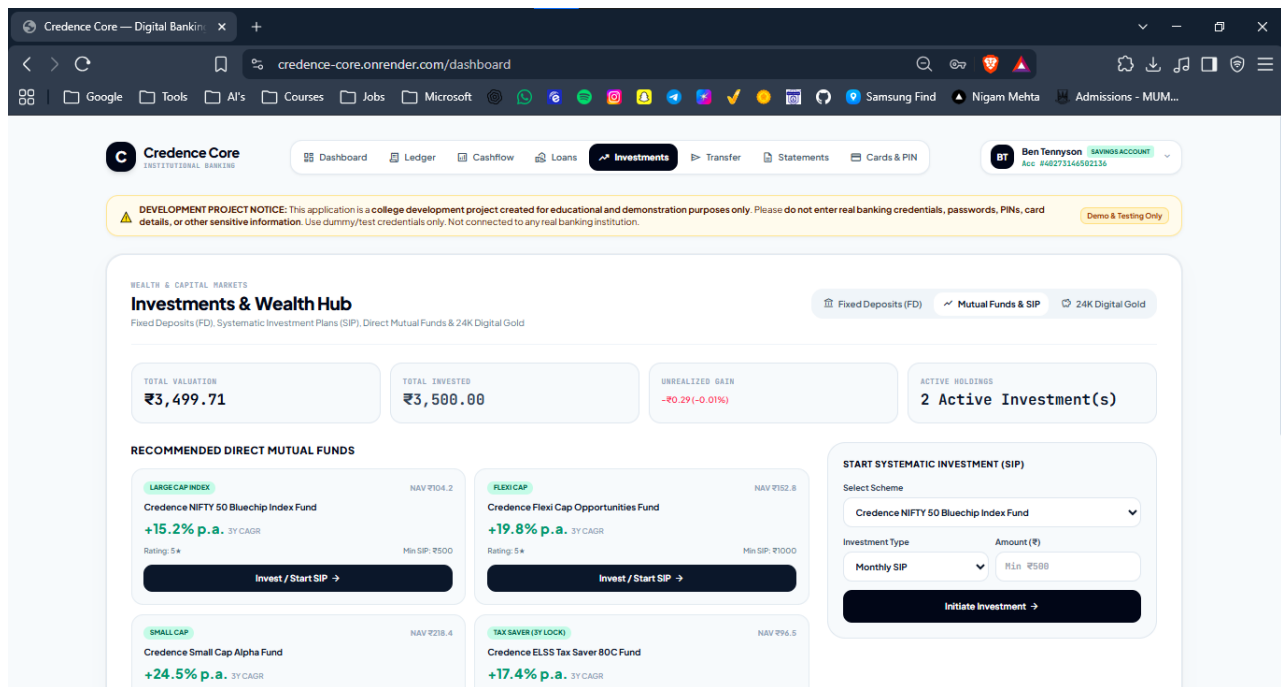


Fig. 9: Investment marketplace view showing recommended mutual-fund cards and investment controls.

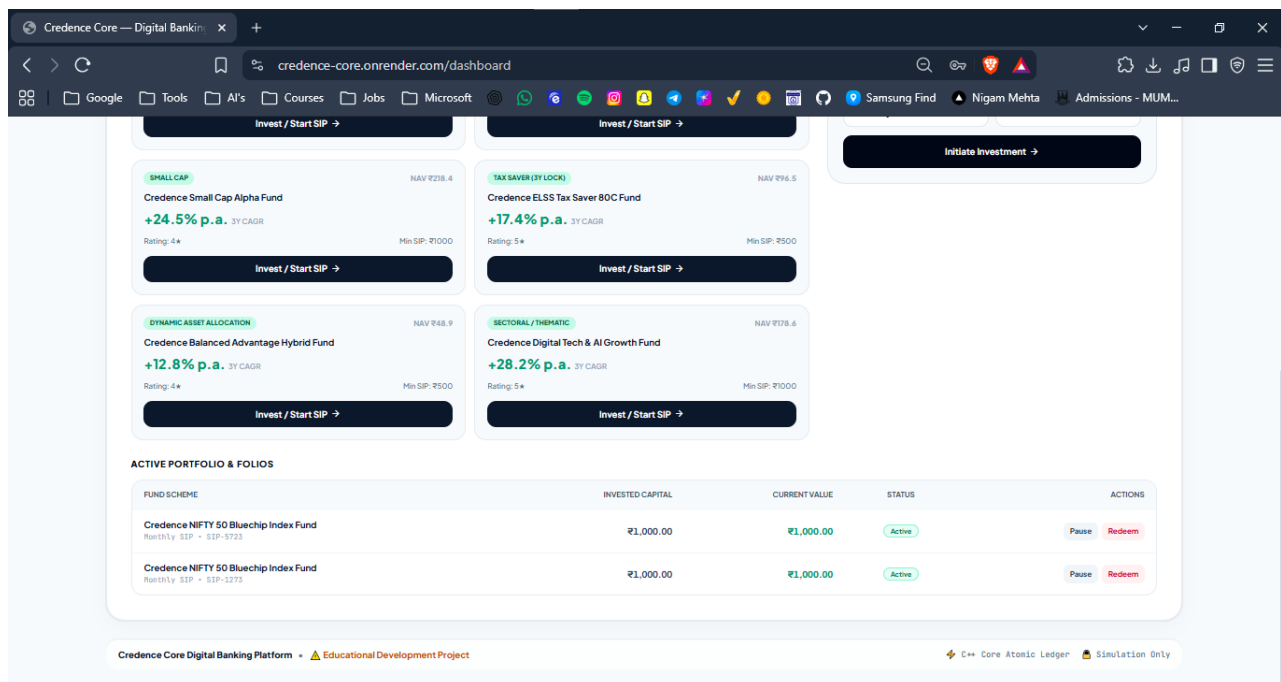


Fig. 10: Active investment portfolio view. The supplied screenshot emphasizes portfolio rows and investment status.

K. Cheque Books and Physical Dispatches

The project also simulates physical banking logistics. The repository includes cheque-book management and front-end dispatch rendering. The dashboard controller combines cheque-book and physical-card collections and renders tracking identifiers and delivery status. This is a useful example of extending a digital banking simulation beyond account

balances into service fulfillment. [3], [17]

V. DEPLOYMENT AND INFRASTRUCTURE

A. Render Deployment

The primary deployment configuration is `render.yaml`. It declares a Python web service named `credence-core`,

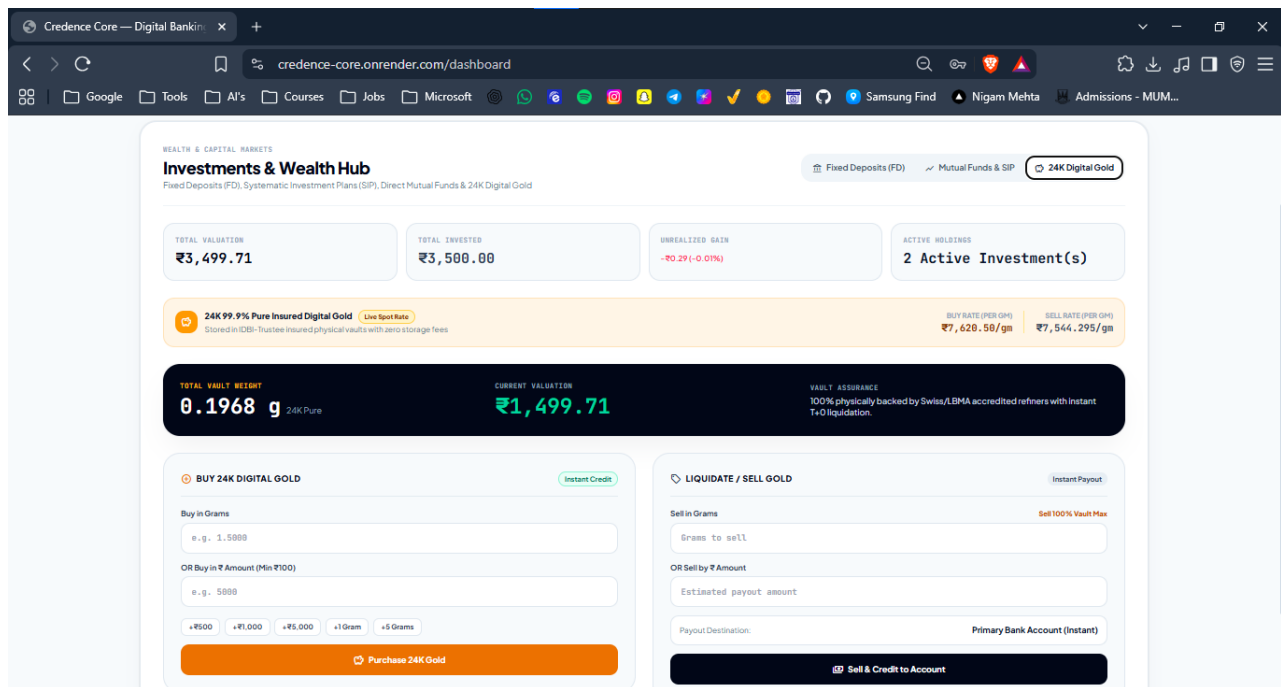


Fig. 11: Additional investment workflow covering gold and investment-entry controls.

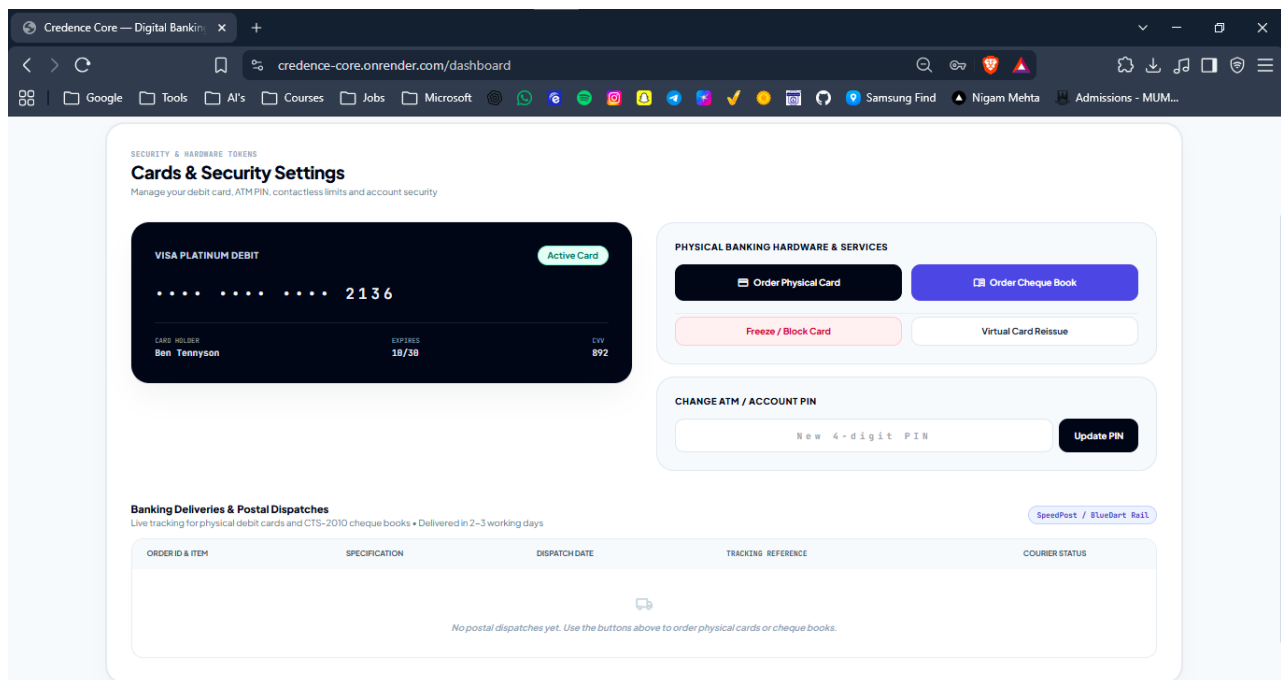


Fig. 12: Cards and Security Settings screen supplied with Credence-Core. The UI presents card information, blocking/security controls and related service records.

installs Python dependencies, compiles the C++ core with g++, and starts the Flask application through Gunicorn. [6]

Listing 3: Render build and start commands from the repository configuration

```
buildCommand: pip install -r requirements.txt && \
  g++ main.cpp account.cpp credit.cpp debit.cpp fd.cpp \
```

```
loan.cpp report.cpp upi.cpp utils.cpp cheque.cpp globals.
  cpp \
  -o bank_system
startCommand: gunicorn flask_app:app
```

This architecture is significant because the cloud container can compile the native banking core at deployment time. The

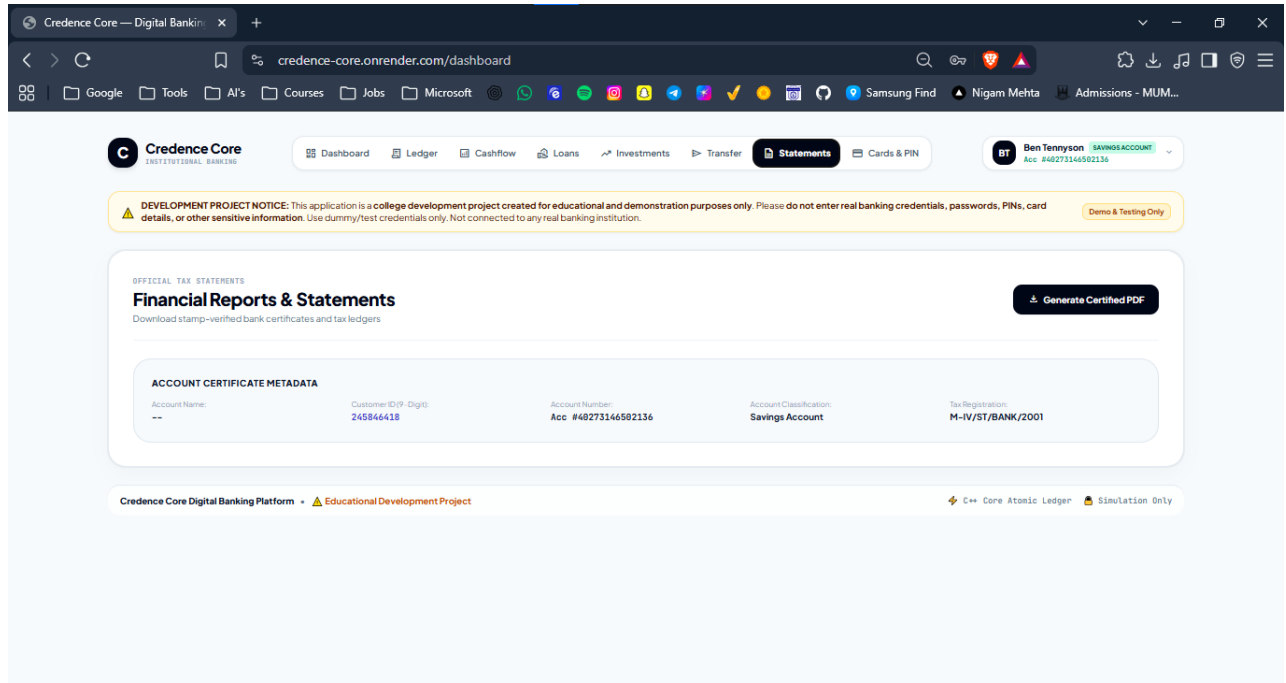


Fig. 13: Financial Reports and Statements interface. The screenshot shows the statement/report workflow available from the banking command deck.

TABLE III: Repository Organization

Path	Responsibility
code.html	Landing/authentication interface.
code(1).html	Unified multi-tab banking command deck.
app.js	Frontend controller, API synchronization and interaction logic.
flask_app.py	Flask application, routes, backend orchestration and PDF/report helpers.
bank.h	C++ core declarations and shared data definitions.
main.cpp	C++ program entry point.
account.cpp	Account creation, numbering, balance and transfers.
credit.cpp/debit.cpp	Credit/debit operations.
fd.cpp	Fixed-deposit operations.
loan.cpp	Loan processing.
upi.cpp	UPI functionality.
cheque.cpp	Cheque-book management.
report.cpp	Reporting logic.
utils.cpp/globals.cpp	Utilities and shared state.
requirements.txt	Python dependencies.
CMakeLists.txt	C++20 build configuration.
Dockerfile	Container image definition.
render.yaml	Render web-service configuration.
vercel.json	Vercel-oriented serverless configuration.

README also documents the expected Render URL, <https://credence-core.onrender.com>. [1], [6]

At the time this report was prepared, a direct request to the public URL returned HTTP 503 from the browsing environment. Therefore, this report does not claim a successful live transaction at report-generation time; the deployment

configuration and the supplied screenshots are treated as the primary evidence of intended runtime behavior.

B. Docker Deployment

The repository includes a Dockerfile based on `python:3.11-slim`. It installs `g++` and build-essential packages, creates an application working directory, and is intended to package the same Flask/C++ hybrid into a reproducible container. [7]

Listing 4: Container build pattern in the repository

```
FROM python:3.11-slim
RUN apt-get update && apt-get install -y --no-install-
recommends \
    g++ build-essential
WORKDIR /app
```

The README documents the corresponding local container commands:

```
docker build -t credence-core .
docker run -p 5000:5000 credence-core
```

C. Vercel Configuration

The repository also contains `vercel.json`, reflecting an alternative serverless deployment path. This is useful for experimentation, although the project architecture is more naturally aligned with a persistent web service because the C++ core is compiled and invoked as part of the application runtime.

D. Performance Considerations

The project is intentionally lightweight: the C++ core avoids a heavyweight database engine, the UI is a single command deck, and JSON persistence is straightforward. However, these choices trade operational scalability for simplicity. File-based state can become a bottleneck under concurrent writes, while a single web process and in-process state are not sufficient for horizontally scaled financial workloads. For production-like scaling, the application would need an external transactional database, centralized session management, background jobs for reporting and dispatch tracking, and explicit concurrency controls.

VI. SETUP AND INSTALLATION GUIDE

A. Prerequisites

A local development environment should provide:

- Python 3.11 or a compatible Python 3 release;
- a C++ compiler with C++20 support;
- CMake 3.20 or newer if the CMake workflow is used;
- Git;
- a modern web browser;
- optional Docker Desktop for container execution.

B. Local Installation

The repository's README gives a Windows-oriented one-click launcher through `run_html_ui.bat`. The manual path is:

```
git clone https://github.com/nigam-30/Credence-Core.git
cd Credence-Core
python -m venv .env
.venv\Scripts\activate
pip install -r requirements.txt
python flask_app.py
```

The browser can then open <http://127.0.0.1:5000/>. The repository also documents direct execution through the included launcher. [1]

C. C++ Build Using CMake

For a clean native build:

```
cmake -S . -B build
cmake --build build --config Release
```

The resulting executable is the `bank_system` target defined by `CMakeLists.txt`. [5]

D. Production-Like Local Run

A production-style local run can use Gunicorn on a Unix-like environment:

```
pip install -r requirements.txt
g++ main.cpp account.cpp credit.cpp debit.cpp fd.cpp \
    loan.cpp report.cpp upi.cpp utils.cpp cheque.cpp globals. \
    cpp \
    -o bank_system
gunicorn flask_app:app
```

This mirrors the Render configuration and is therefore a useful deployment parity check. [6]

VII. CURRENT STATE AND STATUS

A. Working Components Evidenced by the Repository and Screenshots

The current project state demonstrates a broad working surface:

- 1) Institutional landing/authentication interface.
- 2) Unified dashboard and account overview.
- 3) Account creation and customer profile initialization.
- 4) C++ account and transfer logic.
- 5) Transaction ledger and cash-flow analytics UI.
- 6) Loan and FD workflows.
- 7) UPI and payment/service interfaces.
- 8) Investment portfolio, SIP and gold UI/state structures.
- 9) Card/security and dispatch-management screens.
- 10) Financial statement/PDF reporting path.
- 11) Render and Docker deployment definitions.

B. Known Limitations

The most important limitations are architectural rather than cosmetic:

- **Not a real banking system.** The repository itself explicitly limits the application to educational/demo use. [1]
- **Authentication is not production-grade.** Password and PIN handling require a redesign around server-side sessions, password hashing, MFA, rate limiting and authorization middleware.
- **JSON persistence is not transactional.** Concurrent requests and multi-instance deployment can cause consistency problems.
- **Sensitive card information is simulated in application state.** Production card data should never be handled this way.
- **Wildcard CORS is too permissive.** The current configuration allows broad cross-origin access.
- **Demo data exists in initialization code.** This is useful for testing but should be isolated from production configuration.
- **Live deployment availability is not guaranteed.** The public Render endpoint returned 503 during preparation of this report.

C. Code Quality Observations

The modular C++ file split is a strength because account, FD, loan, UPI, cheque and reporting responsibilities are separated. The Flask layer centralizes web concerns, while the frontend controller provides a single place for API synchronization and UI state updates. This structure resembles the modular style seen in the supplied reference reports, where implementation blocks are separated and each is tied to a measurable responsibility. The reference reports emphasize explicit architecture, module-level responsibilities, validation and limitations rather than presenting only screenshots. [?], [?], [?]

There is nevertheless significant room for refactoring. The Flask file is very large, the frontend controller is similarly

monolithic, and several concerns—routing, persistence, business rules, PDF generation and backend orchestration—are combined in one Python module. A next iteration should introduce service modules, data-access functions, typed request/response schemas, centralized error handling, structured logging and automated tests.

VIII. FUTURE SCOPE AND ENGINEERING ROADMAP

A practical next-stage roadmap is:

- 1) Replace JSON persistence with PostgreSQL or another transactional relational database.
- 2) Introduce SQLAlchemy or a dedicated repository/data-access layer.
- 3) Implement server-side sessions or signed JWT access tokens with role-based authorization.
- 4) Replace plaintext/demo authentication fields with Argon2id or bcrypt password hashing and secure secret management.
- 5) Move PIN verification and all transaction authorization to the server.
- 6) Remove wildcard CORS and define an explicit production origin allow-list.
- 7) Separate Flask routes into blueprints for accounts, payments, loans, investments, reports and cards.
- 8) Split `app.js` into feature modules or a component framework when the UI grows further.
- 9) Add unit, integration and end-to-end tests for balance invariants, transfers, loan repayment, FD operations and investment flows.
- 10) Add immutable transaction IDs and an append-only audit trail.
- 11) Introduce idempotency keys for financial POST operations to prevent duplicate submissions.
- 12) Add structured observability: request IDs, metrics, error logging and deployment health checks.
- 13) Use background workers for PDF generation, dispatch tracking and periodic investment calculations.
- 14) Add CI/CD checks for compilation, Python linting, tests and security scanning before deployment.

IX. CONCLUSION

Credence-Core demonstrates a broad and technically interesting full-stack banking simulation by combining a modular C++ ledger engine with Python Flask middleware and a modern browser terminal. The project goes beyond simple deposit/withdrawal functionality: it includes customer identification, transfers, loans, fixed deposits, UPI, cards, cheque services, analytics, investments, digital gold, dispatch tracking and PDF statements. The supplied screenshots confirm a coherent institutional banking interface spanning authentication, dashboard, ledger, analytics, transfers, loans, investments, cards and statements.

From an engineering perspective, the most valuable characteristic is the hybrid architecture. The project shows how a native core can be wrapped by a web API and then consumed by a rich frontend. Its main weaknesses—JSON persistence, simulation-oriented authentication, broad CORS,

client-side PIN checking, demo credentials and monolithic Flask/JavaScript modules—are also clear opportunities for the next development phase. With a transactional database, proper authentication/authorization, stronger testing and service decomposition, the current academic prototype could evolve into a much more robust fintech demonstration platform.

ACKNOWLEDGMENT

The author acknowledges the academic project environment and the use of C++, Python Flask, web technologies, CMake, Docker, Render, ReportLab and associated development tooling in constructing Credence-Core. The report presentation follows the technical depth and structured style of the supplied reference reports, particularly their use of architecture descriptions, implementation details, tables, screenshots, limitations and future scope. [?], [?], [?]

REFERENCES

- [1] N. Mehta, “Credence-Core: Institutional Digital Banking & Wealth Terminal,” GitHub repository, 2026. Available: <https://github.com/nigam-30/Credence-Core>
- [2] N. Mehta, “`flask_app.py`,” Credence-Core source repository, 2026. Available: https://github.com/nigam-30/Credence-Core/blob/main/flask_app.py
- [3] N. Mehta, “`app.js`,” Credence-Core source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/app.js>
- [4] N. Mehta, “`requirements.txt`,” Credence-Core source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/requirements.txt>
- [5] N. Mehta, “`CMakeLists.txt`,” Credence-Core source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/CMakeLists.txt>
- [6] N. Mehta, “`render.yaml`,” Credence-Core source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/render.yaml>
- [7] N. Mehta, “`Dockerfile`,” Credence-Core source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/Dockerfile>
- [8] N. Mehta, “`account.cpp`,” Credence-Core source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/account.cpp>
- [9] N. Mehta, “`fd.cpp`,” Credence-Core source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/fd.cpp>
- [10] N. Mehta, “Credence-Core Flask login and account routes,” source repository, 2026. Available: https://github.com/nigam-30/Credence-Core/blob/main/flask_app.py
- [11] N. Mehta, “Credence-Core Flask UI routes,” source repository, 2026. Available: https://github.com/nigam-30/Credence-Core/blob/main/flask_app.py
- [12] N. Mehta, “Credence-Core account creation implementation,” source repository, 2026. Available: https://github.com/nigam-30/Credence-Core/blob/main/flask_app.py
- [13] N. Mehta, “Credence-Core analytics synchronization,” source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/app.js>
- [14] N. Mehta, “Credence-Core investment synchronization and SIP workflow,” source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/app.js>
- [15] N. Mehta, “Credence-Core balance visibility and PIN workflow,” source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/app.js>
- [16] N. Mehta, “Credence-Core backend execution model,” source repository, 2026. Available: https://github.com/nigam-30/Credence-Core/blob/main/flask_app.py
- [17] N. Mehta, “`cheque.cpp`,” Credence-Core source repository, 2026. Available: <https://github.com/nigam-30/Credence-Core/blob/main/cheque.cpp>